



EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPARTMENT OF PROGRAMMING LANGUAGES
AND COMPILERS

High-Performance Collaborative Text Editor Using HTTP/3 WebTransport

Supervisor:

Kitlei Róbert László

Assistant Professor

Author:

Omar Abdelkader

Computer Science BSc

Budapest, 2026

EÖTVÖS LORÁND UNIVERSITY
FACULTY OF INFORMATICS

Thesis Registration Form

Attention! The deadline for amendment is 1 February in case of final exam in June and 31 August in case of final exam in January.

Student's Data:

Student's Name: Abdelkader Omar Ahmed Ibrahim

Student's Neptun code: WQ54YY

Educational Information:

Training programme: Computer Science BSc

I have an internal supervisor

Internal Supervisor's Name: *Kitlei Robert Laszlo*

Supervisor's Home Institution: *Department of Programming Languages and Compilers*

Address of Supervisor's Home Institution: *1117, Budapest, Pázmány Péter sétány 1/C.*

Supervisor's Position and Degree: *MSc in Computer Science*

Thesis Title: Real-Time Collaborative Session Sharing Extension for Visual Studio Code

Topic of the Thesis:

(Upon consulting with your supervisor, give a 150-300-word-long synopsis of your planned thesis.)

The goal of my thesis project is to develop a real-time session-sharing extension for Visual Studio Code that allows multiple users to edit the same file simultaneously, similar to Live Share, but as an extension, I will explore the benefits that HTTP/3 can bring to this application by adopting the WebTransport API. My goal is to enable multiple developers to work simultaneously on the same codebase with immediate responsiveness, which can be achieved by implementing a consistency model resulting in strong eventual consistency using the optimistic replication approach. The application will maintain cursor consistency that will broadcast the updates of the user's cursor position, text edition, and presented data. As Visual Studio utilizes JavaScript for its extensions, I have selected Node.js for the backend to maintain a unified JavaScript ecosystem across the entire stack. The thesis will also include testing of the API under a simple interface to measure the performance of the WebTransport layer.

Budapest, 2026. 01. 30.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Assistant Professor Kitlei Róbert László, for his continuous guidance, technical insights, and patience throughout the development of this project. I am also thankful for his suggestion to develop it as a Visual Studio Code extension. This advice gave the project a clear, practical focus. I would like to thank my brother for his constant feedback on my progress and support throughout the process of my thesis which pushed me to solve major issues that I encountered.

Contents

Acknowledgements	2
1 Introduction	3
1.1 Motivation	3
1.2 Summary	4
2 User Documentation	5
2.1 Role Concepts	5
2.2 System Prerequisites	6
2.2.1 Hardware Requirements	6
2.2.2 Software Requirements	6
2.2.3 Network Requirements	7
2.3 Practical Usage Guide	7
2.3.1 Requirements Installations	7
2.3.2 setting up the project locally	8
2.3.3 User Guide	9
3 Developer Documentation	17
3.1 The Architectural Problem Statement	18
3.2 The Architectural Solution	19
3.2.1 Bypassing Environment Restrictions via Webview Proxy	20
3.2.2 Resolving Concurrency with CRDTs	21
3.2.3 Synchronizing the Graphical User Interface	22
3.2.4 Overcoming Security Sandboxing via Cloud Deployment	22
3.3 Environment Requirements	23
3.4 Run and Deployment	23
3.4.1 Local Front-end Execution	23
3.4.2 Cloud Back-end Persistence	23

3.5	Project Structure	24
3.6	Data Model (CRDT)	25
3.7	Front-end Implementation	26
3.8	Back-end Implementation	27
3.9	Architectural and Security Challenges	28
3.9.1	The Sandbox and Private Network Access (PNA)	28
3.9.2	Presence Data and Ephemeral State Garbage Collection	29
3.9.3	CRDT Tombstone Accumulation and Memory Bloat	30
3.9.4	Middleware Lifecycle and Zombie Processes	31
3.10	Security and Access Control	31
3.10.1	Role Specifications	31
3.10.2	Room ID Cryptography	32
3.11	Testing and Validation	33
3.11.1	Unit Testing (CRDT State)	33
3.11.2	Integration Testing (The Proxy Pipeline)	33
3.11.3	Manual UI and Network Testing	33
3.12	Chapter Conclusion	36
	Conclusion	37

Chapter 1

Introduction

1.1 Motivation

One of the biggest challenges in modern real-time collaborative applications is the reliance on traditional TCP/IP transport stacks. Software developers today mostly work in teams using shared, real-time code editors, but TCP was not built for fast, multiplayer typing. Because TCP enforces strict sequential delivery, it is highly exposed to application-level Head-of-Line (HoL) blocking. If a single packet containing heavy document state data is dropped, the entire stream halts until the packet is re-transmitted. For the end-user, this results in artificial UI lag, where their collaborator’s cursor freezes and stutters on the screen simply because an unrelated background text packet was delayed. Analytical benchmarks demonstrate that under a minimal 0.5% packet loss, TCP latency can exponentially degrade due to these re-transmission delays, completely stalling the network [bondar2025].

One of the most important solutions created in recent years is Quick UDP Internet Connections (QUIC), a transport protocol developed by Google. It surpasses many of the limitations straining traditional TCP/UDP transport under modern demands, combining the low-latency benefits of UDP with the reliability and flow control of TCP. Unlike older protocols, QUIC has built-in encryption and does not support unsecured connections. It also features multi-threading, as well as 0-RTT (zero latency) and 1-RTT (one round trip time) connection establishments (see in Figure 1.1). This significantly speeds up the initial handshake, showcasing how QUIC-based architectures maintain high efficiency in real-time data transmission [antonova2025].

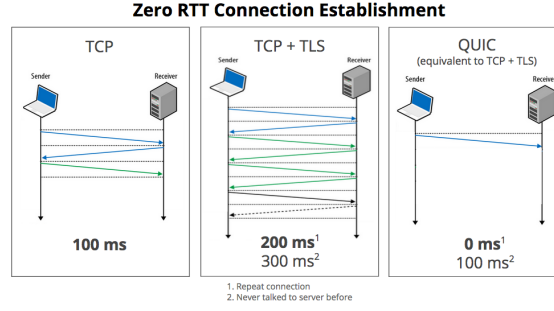


Figure 1.1: Comparison of connection establishment times for different network protocols.[chromium2015]

Therefore, this thesis proposes a practical software solution that tries to solve TCP bottlenecks: a high-performance collaborative text editor built as a Visual Studio Code extension. When deciding on the architecture, older WebSocket technologies were rejected because they still suffer from TCP’s sequential delivery limits. Instead, this program replaces them with HTTP/3 WebTransport to benefit from multiplexed streams, unreliable datagrams, and the elimination of HoL blocking. By combining the Yjs CRDT (Conflict-free Replicated Data Type) framework with WebTransport, this project delivers a fast, multi-file shared workspace.

1.2 Summary

In chapter 2, the user documentation is provided. It gives a clear, step-by-step guide for end-users, explaining how to install the extension, establish a secure connection, use the real-time shared workspace and benefit from the extension’s features.

Following this, chapter 3 contains the developer documentation. This chapter discusses all the critical technical details about the development of the application. It illustrates the general architecture, covering both the Visual Studio Code Webview front-end and the Node.js back-end. Importantly, it documents the key design decisions made during development, including the reasoning behind deploying the server to a cloud environment to bypass strict local network sandboxes and the journey behind it.

Finally, in (chapter 3.12) a conclusion is attached, summarizing the solution and listing out the main outcomes from the project. It also explores potential future improvements from a developer’s perspective.

Chapter 2

User Documentation

Modern software development increasingly relies on real-time collaboration, but developers often face synchronization delays and interface interruptions when using traditional tools. To address these limitations, this project introduces a Visual Studio Code (VS Code) Extension designed to mitigate network friction and provide a more responsive collaborative environment directly inside the developer's native editor. This chapter serves as the user manual for the extension, detailing its purpose, system prerequisites, and practical usage. Because the installation and operation of this tool require navigating through command-line interfaces (terminals) and the VS Code Extension Development Host, this guide is specifically intended for end-users who possess a foundational background in programming and software development environments.

2.1 Role Concepts

Unlike centralized applications with permission hierarchies (e.g., administrators, managers), this peer-to-peer architecture simplifies users into two functional roles:

- **Host Developer:** The user who initializes the collaborative session, generating a unique Room ID and sharing their local workspace state.
- **Guest Developer:** A user who joins an active session using a provided Room ID. They receive the shared workspace files and can edit it synchronously.

2.2 System Prerequisites

To ensure a smooth collaborative editing experience without degrading local development performance, the user's system must meet specific criteria.

2.2.1 Hardware Requirements

Component	Minimum	Recommended
Processor	64-bit dual-core CPU	Quad-core CPU (Intel Core i5 / AMD Ryzen 5 or faster)
Memory	4 GB RAM	8 GB RAM or more
Storage	2 GB available space	SSD with 5 GB available space
Display	1280 x 800 resolution	1920 x 1080 (Full HD) resolution

Table 2.1: Hardware Requirements

2.2.2 Software Requirements

Before installing the extension, ensure you have the following software installed:

- **Operating System:** Windows 10/11 (with WSL2 enabled), macOS, or Linux.
- **Editor:** Visual Studio Code (version 1.80 or newer).
- **Git:** This is a version control system used to download (or "clone") the extension's source code directly from its online repository to your local machine.
- **NVM:** A tool used to install and manage multiple versions of Node.js on a single machine. Because this extension relies on modern JavaScript syntax for its WebTransport networking protocol, NVM ensures the user can easily install the required, up-to-date version of Node.js (v20+) without conflicting with other local development projects.
- **Node.js:** This is a runtime environment that executes JavaScript code outside of a web browser. Because the Visual Studio Code extension relies on background JavaScript processes to handle the real-time collaboration logic, Node.js must be installed to power the extension's engine.

- **Npm:** Bundled automatically with Node.js, this is the tool used to securely download and manage external code libraries. The user must run this tool once during setup to fetch all third-party dependencies required by the extension, such as the Yjs framework and the WebTransport networking modules.

2.2.3 Network Requirements

Because this extension uses a modern peer-to-peer network architecture, it handles poor network conditions (like high latency). However, it requires specific network permissions to connect:

- **Firewall Permissions:** Your local network and firewall must allow outbound UDP traffic on Port 4433. (If you are on a strict corporate or university Wi-Fi network, this port may be blocked).
- **Bandwidth:** Minimum 100 Kbps connection.
- **Latency:** Under 150 ms is recommended for the best real-time cursor tracking, although the text synchronization will remain stable even on slower connections.

2.3 Practical Usage Guide

The following sections explain the step-by-step usage of the extension.

2.3.1 Requirements Installations

1. **Installing Git:** If you don't have Git installed, download it from Git's official website. Follow the installation instructions based on your OS.
2. **Install VS Code:** If you don't have VS Code installed, you can download and set up the application from VS Code's official website. Follow the installation instructions based on your OS.
3. **Installing NVM:** Open a terminal and run:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh  
| bash
```

4. Installing Node.js:

1. Install Node.js and verify the installation by opening a terminal/command prompt and running those commands:

```
nvm install node  
node -v
```

5. **Npm(node package manager):** It will be installed automatically with node.js, but you can verify the installation by running this command which will show you the current version:

```
npm -v
```

2.3.2 setting up the project locally

1. **Cloning The Project:** Download or clone the source code repository to your local computer to the desired directory. For cloning open terminal and use this Git command:

```
git clone https://github.com/maroahm/vscode-crdt-collaborative-editor
```

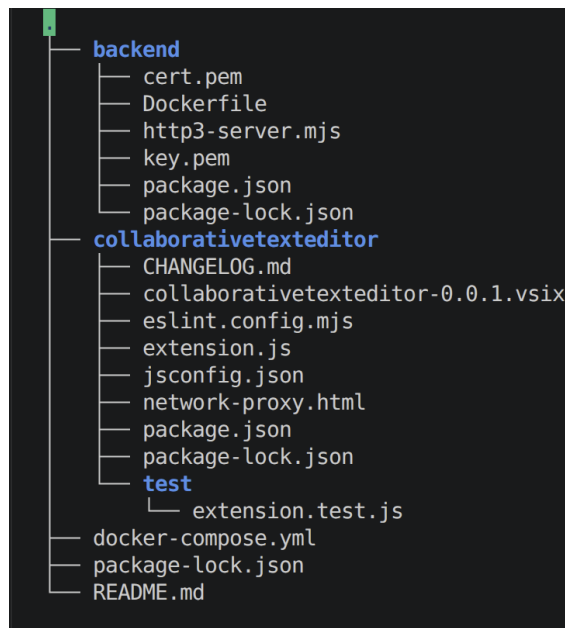


Figure 2.1: Image description of the folder structure of the cloned repository.

2. **Installing Dependencies:** Open your terminal and navigate to the cloned repository and go to the extension folder and installing the required dependencies by running:

```
cd vscode-crdt-collaborative-editor/colaborativetexteditor
npm install
```

3. **Launch the Extension Development Host:** To safely test and interact with the extension without affecting your primary VS Code editor settings, open a new terminal window, navigate to the extension's root directory and make sure you are in the `colaborativetexteditor` directory, and execute the following command:

```
code -extensionDevelopmentPath="$(pwd)" -user-data-dir="tmp/vscode-client-A" de-client-A"
```

***Note:** You can change the "tmp/vscode-client-A" path to any directory name where you would like to save your isolated Extension Host data.*

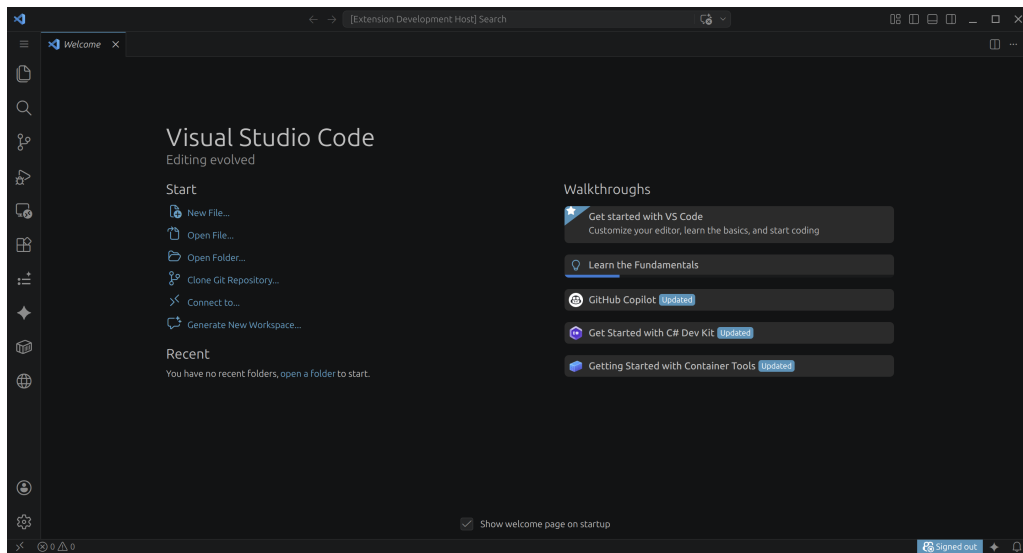


Figure 2.2: VS Code development host

2.3.3 User Guide

This sub-section provides a step-by-step visual guide for each feature of the collaborative extension.

1. **Hosting/Joining/disconnecting from a session:**

- (a) Inside your VS Code development host, Press **Ctrl+Shift+P** to open the Command Palette (see in Figure 2.3), and type "**HTTP/3 Yjs:Manage Session**"

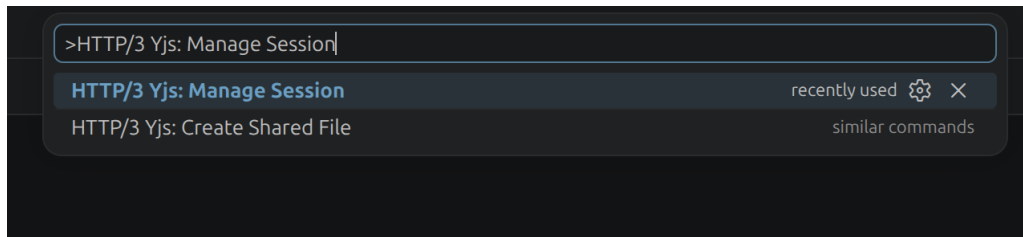


Figure 2.3: Searching command pallet

- (b) After choosing it, you will see two options "**create new room**"(host a session) or "**join existing room**", as shown in Figure 2.4.

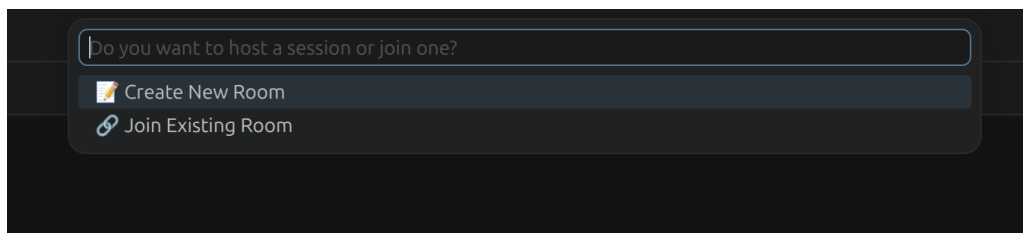


Figure 2.4: Hosting a session or joining one

- (c) then you have to enter your name if you are hosting a session, as shown in Figure 2.5.

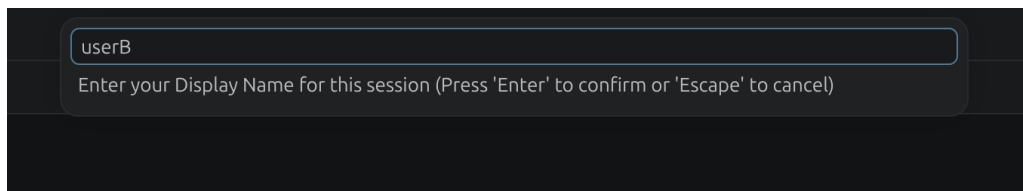


Figure 2.5: Entering a name

- (d) A session will be created showing the network proxy and the session id, as shown in Figure 2.6.

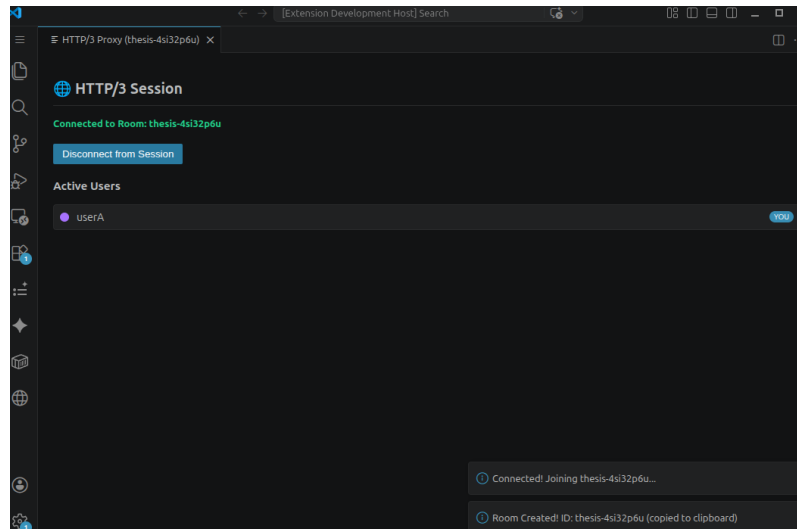


Figure 2.6: Hosting a session

- (e) if you are joining a session you have to enter both your name and the provided session code(e.g, `thesis_abc12345`) provided by the host, as shown in Figure 2.7.

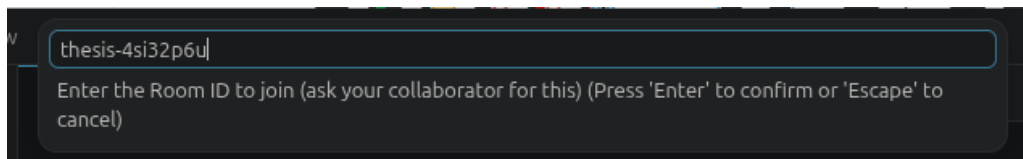


Figure 2.7: Using provided session id

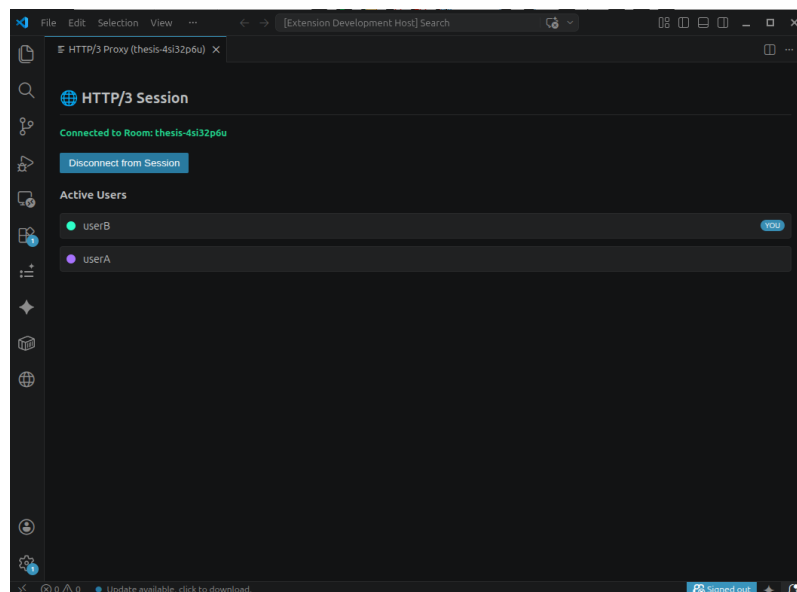


Figure 2.8: Joining a session as a guest user

- (f) To disconnect from the sever, you just have to press the disconnect button, as shown in Figure 2.9 and other users will see the new active users (see in Figure 2.11)

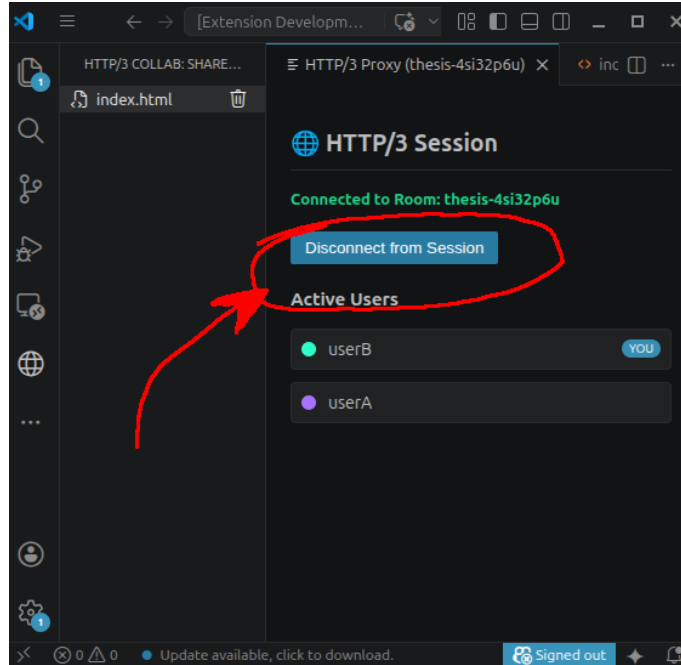


Figure 2.9: Disconnect button

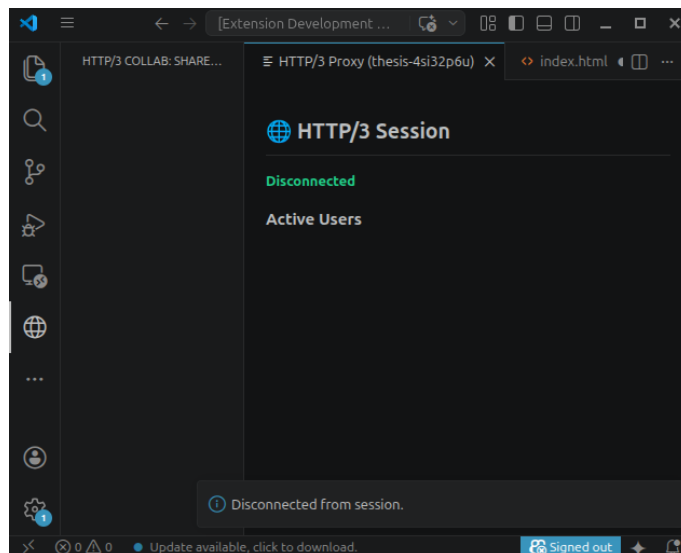


Figure 2.10: Disconnecting from a session

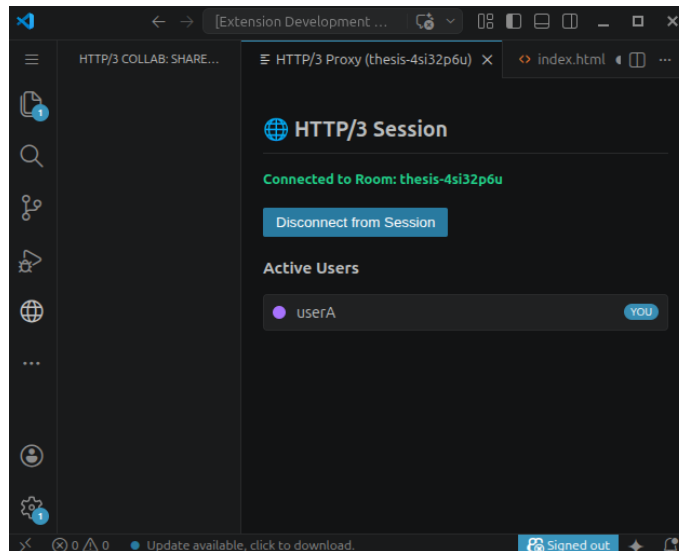


Figure 2.11: Other user's screen updates

2. Interacting with the shared file explorer:

- (a) To open the file explorer, where you would be able to see the files shared in during the session, you need to open the command palette and type “View: Show HTTP/3 Collab”, as shown in Figure 2.12.

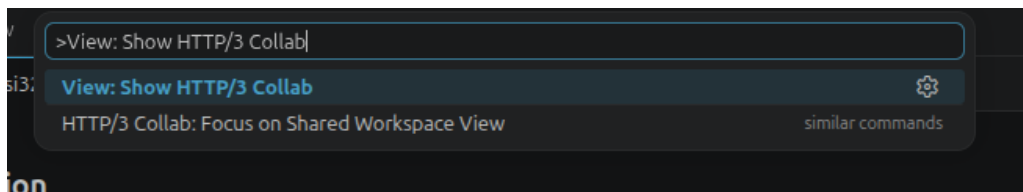


Figure 2.12: Selecting view file explorer command

- (b) The file explorer tab appears, as shown in Figure 2.13.

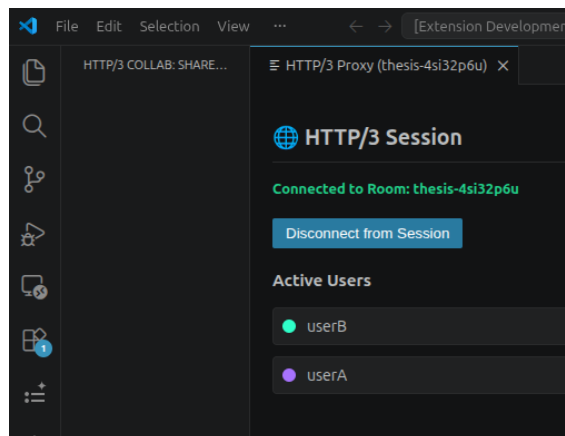


Figure 2.13: Shared file explorer tab

- (c) To create a shared file you have to enter "HTTP/3 Yjs: Create Shared File" into the command palette. (see in Figure 2.14)

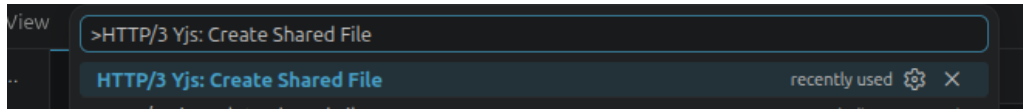


Figure 2.14: Create shared file command

3. Then enter the name of the file. (see in Figure 2.15)

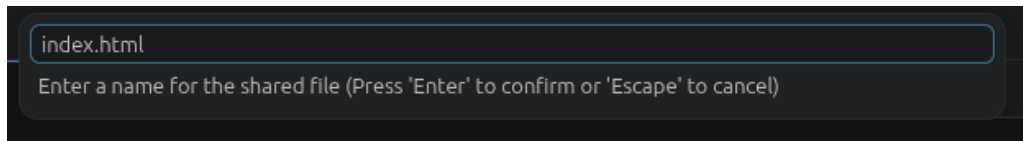


Figure 2.15: Entering file name

4. File created in both user's screens, as shown in Figure 2.16 and Figure 2.17

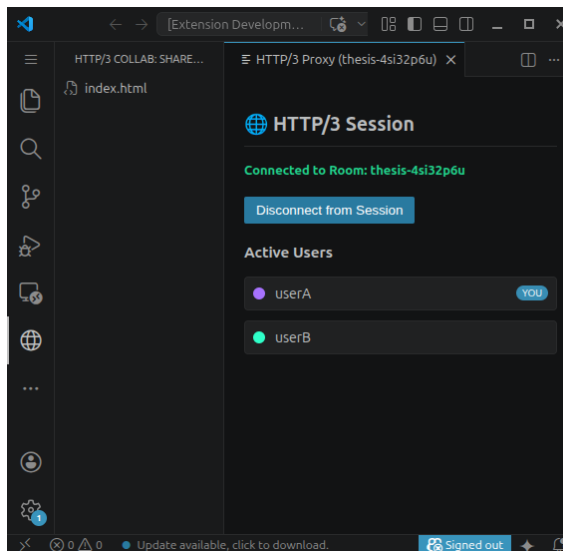


Figure 2.16: File showing in userA's file explorer

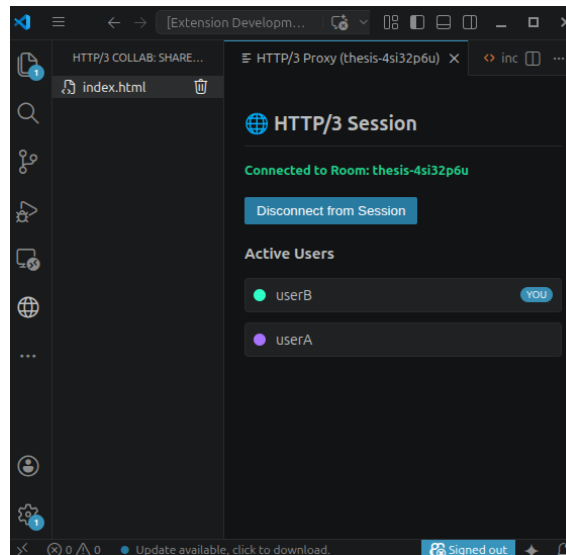


Figure 2.17: File showing in userB's file explorer

5. Text and cursor consistency for both users when using the file, as shown in Figure 2.18 and Figure 2.19

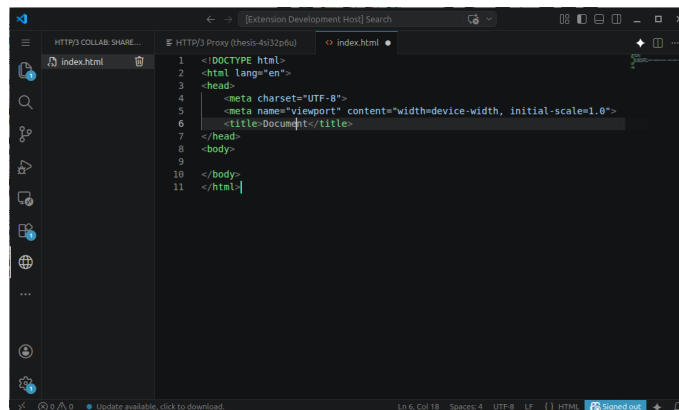


Figure 2.18: userA's screen

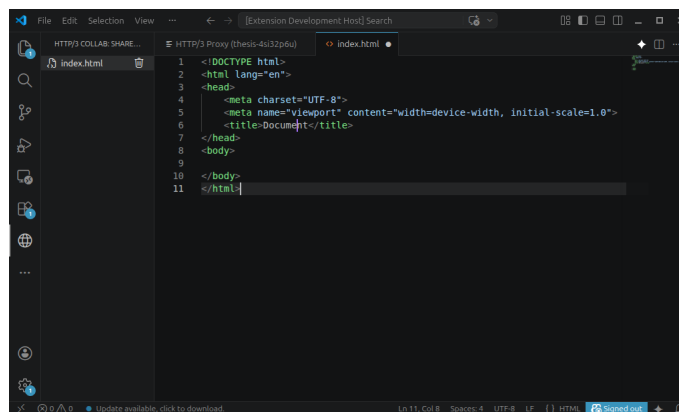


Figure 2.19: userB's screen

6. To delete a file, press the trash icon next to the files name, as shown in Figure 2.20 and it will get deleted for all users. (see in Figure 2.21 and Figure 2.22)

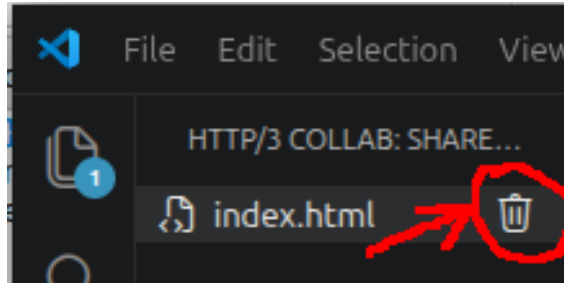


Figure 2.20: Deleting button

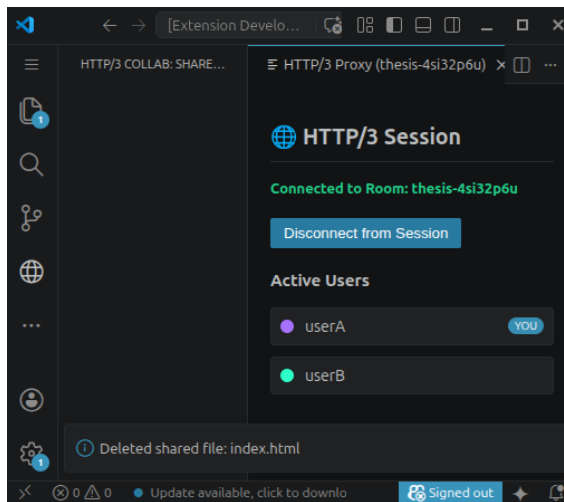


Figure 2.21: userA's screen

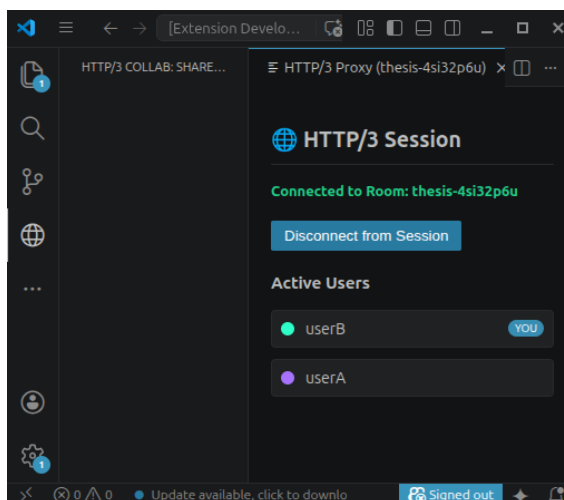


Figure 2.22: userB's screen

Chapter 3

Developer Documentation

This chapter is dedicated to developers and provides detailed insights into the technical infrastructure, architectural decisions, and implementation specifications of the collaborative extension. Section 3.1 discusses the structural limitations of standard collaborative environments, detailing how transport protocols like TCP and UDP create bottlenecks that necessitate a new architectural approach and the difficulties involved in choosing a fast, reliable collaborative architecture. Section 3.2 details the current code implementation and its architectural foundation, explaining the specific technologies introduced to overcome these faced issues. Section 3.3 lists the required development environment. Section 3.4 provides instructions for local execution and cloud deployment. Section 3.5 gives an overview of the architecture, followed by section 3.6, section 3.7, and section 3.8 which document the CRDT data model, front end, and back end respectively, alongside detailed justifications for the technical decisions made throughout the development of the program. Then comes section 3.9, that outlines the critical security and network challenges resolved during development. For section 3.11, it concluded all the important tests that were made on the critical parts of the program. Finally, section 3.12 wraps up the chapter by summarizing how these architectural shifts successfully bypass the initial technical limitations, providing future developers with the foundation needed to extend the project.

3.1 The Architectural Problem Statement

Developing a real-time collaborative text editor inside Visual Studio Code presented four significant technical challenges that standard client-server architectures couldn't resolve.

1. **Performance and Latency (TCP Limitations):** Traditional collaborative tools rely on WebSockets, which establish a single, shared TCP connection. Because TCP enforces strict packet ordering across this single pipeline, all network traffic from critical text edits to temporary mouse cursor coordinates are forced into a single pipeline. If just one network packet is dropped, it triggers Head-of-Line (HoL) blocking, halting the entire data stream until that specific packet is found and successfully re-transmitted. In a real-time editor, this means a minor network interruption will freeze remote cursors and the entire user interface while the system waits for the TCP pipeline to clear, creating a disruptive user experience.
2. **Environment Restrictions:** The modern network solution to TCP's latency limitations is WebTransport (running over UDP/HTTP3). However, the core Node.js environment that natively powers VS Code extensions does not yet support the experimental WebTransport API.
3. **Security Sandboxing and Local Testing:** To use WebTransport inside VS Code, the extension must rely on a hidden Chromium browser window (a Webview). However, this triggers strict, unavoidable security rules. First, Chromium's Private Network Access (PNA) policy kept blocking web scripts from connecting to local networks (like `localhost`), which makes local testing impossible. Second, WebTransport requires strict TLS 1.3 encryption, but the hidden window automatically rejects the self-signed security certificates typically used for local development because there is no user interface to manually bypass the warning.
4. **Protocol Reliability vs. Speed (UDP Limitations):** While replacing TCP for raw UDP (User Datagram Protocol) resolves Head-of-Line blocking, it leads to other inconsistencies in the shared document. UDP is "fire-and-forget" which means that the sender transmits data packets into the network without

ever verifying if the recipient actually received them, so it's an unreliable and unordered protocol. It does not guarantee that a packet will arrive, nor does it guarantee packets will arrive in the sequence they were sent. In a collaborative code editor, a dropped raw UDP packet means a permanently lost line of code, and an out-of-order packet which could apply a deletion before an insertion, instantly corrupting the document state.

5. **Concurrency and Data Consistency:** In a real-time environment, multiple peers frequently change the document simultaneously. Consider a scenario where User A inserts text at indexes 1 through 5, User B simultaneously inserts text at indexes 5 through 10, and User C deletes characters from indexes 3 through 6. If these operations are transmitted over a network and applied out-of-order using absolute indexing, the document state will eventually corrupt, leaving each user with a completely different final text. The architecture requires a mathematical model capable of guaranteeing a Strong Consistency model ensuring all peers see the exact same final document without relying on a centralized sequencing server.

3.2 The Architectural Solution

To overcome the limitations outlined in section 3.1, I implemented a decoupled, which means separating the front end and back end and connecting via VS Code API, hybrid architecture. The system tries to solve the existing problems using three distinct environments: a UI Extension Host, a hidden Chromium Webview Proxy, and a remote cloud HTTP/3 server.

The solution is achieved through a combination of three client-side Object-Oriented classes and a strategic public back-end deployment, each engineered to resolve a specific architectural challenge. And that is represented in Figure 3.1, that will be explained in the following subsections.

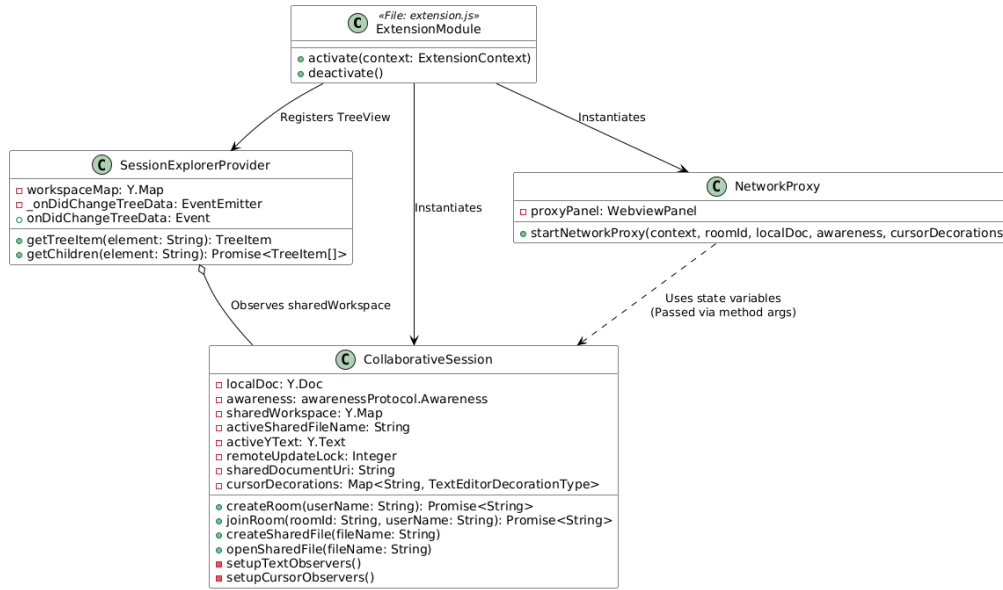


Figure 3.1: Extension class diagram

3.2.1 Bypassing Environment Restrictions via Webview Proxy

To solve the issue of the Node.js Extension Host lacking native WebTransport support, the architecture employs a bridge between them.

The NetworkProxy Class: Because Visual Studio Code is powered by the Electron framework, its execution environment is divided between a Node.js back end and a Chromium front end. To take advantage of this front-end capability, This class instantiates a hidden, sandboxed HTML Webview panel. By utilizing the Webview's Chromium browser engine, the extension can negotiate WebTransport connections to the cloud server, bypassing Node.js limitations.

- **startNetworkProxy(...):** The entry point for network communication. It executes `vscode.window.createWebviewPanel` and establishes an asynchronous pipeline between the extension core and the hidden Chromium tab to pass messages.
- **Message Interception (onDidReceiveMessage):** Acts as a local router. It listens for 'incoming-msg' emitted by the Webview's WebTransport stream. If the message is a 'doc' type, it routes the mathematical update to the local Yjs document; if it is an 'awareness' type, it routes the remote cursor coordinates.

3.2.2 Resolving Concurrency with CRDTs

To address the data corruption and indexing issues inherent in real-time collaborative editing over UDP, the system relies on Conflict-Free Replicated Data Types (CRDTs) to guarantee Strong Eventual Consistency (SEC) without a central sequencing server. Unlike strong consistency models, it is possible to implement SEC in decentralised settings without any central server or leader, and it allows local execution at each node to proceed without waiting for communication with other nodes.[gomes2017]

The CollaborativeSession Class: This class serves as the central engine of the extension which holds the local instance of the Yjs document and translates VS Code editor events into mathematically mergeable data points.

- **setupTextObservers():** Hooks into `onDidChangeTextDocument`. It calculates exact character insertions and deletions, translating them into Yjs operations before committing them to the local `Y.Text` object to trigger a network broadcast. It utilizes a `remoteUpdateLock` integer to prevent infinite feedback loops when remote text is programmatically injected.
- **setupCursorObservers():** Translates absolute editor coordinates into Yjs relative positions (`Y.createRelativePositionFromTypeIndex`). This ensures that if a remote collaborator deletes earlier lines, the local cursor shifts rather than highlighting incorrect text.
- **createRoom(userName):** Initializes the collaborative environment by generating a unique cryptographic Room ID (`thesis-[hash]`) and writing the initial state of the active editor into the shared `Y.Map` workspace.
- **joinRoom(roomId, userName):** Connects a guest client to an active session. It utilizes the provided cryptographic Room ID to establish the network proxy connection and initializes the synchronization process to align the local `Y.Doc` state with the remote workspace.
- **createSharedFile(fileName):** Expands the collaborative workspace by instantiating a new, empty `Y.Text` object and assigning it to the specified file name key within the root `Y.Map`. This structural workspace is automatically broadcasted to all connected peers.

- `openSharedFile(fileName)`: Handles the programmatic UI transition when a developer switches active files. It detaches local network observers from the previous file, retrieves the newly requested `Y.Text` structure from the `Y.Map`, and populates the VS Code editor while utilizing state comparison checks to prevent initialization echo loops from other files.

3.2.3 Synchronizing the Graphical User Interface

The next challenge is abstracting this complex mathematical state into a standard, interactive interface for the developer.

The SessionExplorerProvider Class: This class is responsible for rendering the graphical user interface (GUI) inside the VS Code sidebar. It implements the `vscode.TreeDataProvider` interface to map the collaborative workspace's internal CRDT state to a visual file tree.

- `getChildren(element)`: Reads the active keys from the shared `Y.Map` (the CRDT file system) and dynamically converts them into an array of visual `TreeItem` objects.
- `getTreeItem(element)`: Configures the interactive properties of each file in the sidebar. It attaches the `collaborativetexteditor.openSharedFile` command to handle user clicks and assigns a `contextValue` of `'sharedFile'`, exposing the custom file deletion UI.

3.2.4 Overcoming Security Sandboxing via Cloud Deployment

To resolve the strict Private Network Access (PNA) policies and TLS certificate restrictions imposed by the hidden Chromium Webview, the architecture abandons local back-end execution.

The Remote HTTP/3 Server: Instead of running the Node.js WebTransport server on a local development machine (`localhost`), the back end is deployed to a public cloud infrastructure (`DigitalOcean`). By provisioning the server with a static, public IP address and a trusted TLS 1.3 certificate (via `Let's Encrypt`[`letsencrypt`]), the deployment inherently satisfies Chromium's security sandbox requirements. which caused a blocked local script to run on a secure internet

connection, enabling the Webview Proxy to successfully complete the WebTransport handshake, that was continuously failing.

3.3 Environment Requirements

The application uses Visual Studio Code for front-end extension development and a Node.js environment for the back-end WebTransport server.

For the back end to successfully parse modern JavaScript syntax (specifically ES modules and JSON import assertions like `with { type: 'json' }`), the environment must run **Node.js version 20.10 or higher**. During initial development, older versions (v18) resulted in strict syntax errors when loading cryptographic certificates.

Required Tools:

- **Node.js (v20+)**: The core JavaScript runtime.
- **npm**: The package manager used to install the `yjs` and `@fails-components/webtransport` libraries.
- **Visual Studio Code (v1.80+)**: Provides the Extension API necessary to build the UI and editor interactions.

3.4 Run and Deployment

3.4.1 Local Front-end Execution

To test the front-end extension, developers must compile the code and launch an isolated VS Code environment (the Extension Development Host). This prevents experimental code from crashing the developer's primary editor.

```
1 code --extensionDevelopmentPath="$(pwd)" --user-data-dir="tmp /  
    vscode-client-A"
```

3.4.2 Cloud Back-end Persistence

The back-end server is deployed on a Linux (Ubuntu) cloud infrastructure. To ensure the HTTP/3 server runs continuously without terminating when the SSH connection closes, run the following command prompts.

```
1  # Install PM2 globally
2  npm install -g pm2
3
4  # Start the server and ensure it survives a system reboot
5  pm2 start http3-server.mjs --name yjs-server
6  pm2 save
7  pm2 startup
```

Thesis Decision Log: Process Management

- **The Scenario:** Ensuring the connectivity of the collaborative back end.
- **The Decision:** Implemented pm2 as a background process manager on the cloud droplet.
- **Why this was chosen:** pm2 provides automatic restarts and process monitoring.
- **Why the alternative was rejected:** Running node http3-server.mjs directly in the terminal would cause the server to terminate immediately upon closing the SSH connection.

3.5 Project Structure

The system architecture is divided into three decoupled components to bypass framework limitations.

1. **VS Code Extension (Front end):** Handles the user interface, intercepts typing events, and draws remote cursors using the VS Code API.
2. **Webview Network Proxy (Middleware):** An embedded HTML/JS panel that manages the complex HTTP/3 network handshakes.
3. **Node.js Server (Back end):** A central router that accepts QUIC connections and multiplexes data streams between peers.

Thesis Decision Log: The Webview Proxy

- **The Scenario:** Establishing a WebTransport connection from within a VS Code Extension.
- **The Decision:** Created a hidden Webview panel to act as a network proxy.
- **Why this was chosen:** The Node.js environment powering VS Code extensions lacks native support for the experimental WebTransport API. However, the embedded Chromium browser engine inside a VS Code Webview does support it. The proxy establishes the connection and passes binary data back to the extension via `postMessage`.

3.6 Data Model (CRDT)

Unlike traditional centralized databases (SQL/NoSQL), this application uses a Conflict-Free Replicated Data Type (CRDT) to manage state. The **Yjs** framework is utilized to ensure that concurrent edits by multiple users merge mathematically without conflicts.

The state is structured hierarchically:

- **Y.Doc (The Root):** The primary document container holding all collaborative state.
- **Y.Map ('workspace-files'):** A shared dictionary acting as the file system. The keys are file names (Strings), and the values are **Y.Text** objects.
- **Y.Text:** An array like structure representing the actual code inside a file. It tracks inserts and deletes character by character.
- **Awareness Protocol:** An ephemeral state manager used to track non-permanent data, specifically mouse cursor coordinates and user colors.

Thesis Decision Log: Concurrency Control Architecture

- **The Scenario:** Selecting the core mathematical engine to handle concurrent document changes across multiple distributed clients.

- **The Decision:** Implemented Conflict-Free Replicated Data Types (CRDTs), specifically utilizing the Yjs framework, opting against both server-centric and distributed Operational Transformation (OT) solutions.
- **Why this was chosen:** CRDTs are mathematically designed to achieve Strong Eventual Consistency (SEC) in decentralized environments. Recent research highlights that operation-based CRDTs effectively function via an "indirect transformation approach" [sun2020], replacing absolute document indexes with unique mathematical object identifiers. This allows nodes to merge state updates locally without a central traffic cop.
- **Why the alternative was rejected:** While traditional OT (utilized by Google Docs) was rejected because it necessitates a central server, computer science literature confirms that fully Distributed OT algorithms do exist for peer-to-peer editing [sun2020]. However, implementing complex distributed OT algorithms from scratch or maintaining vector clocks manually within the VS Code Extension Host would introduce higher developmental complexity. Furthermore, building a custom synchronization engine would shift focus away from the core goal of this project, which is to compare the new QUIC protocol against traditional network architectures. Yjs was chosen because it provides a production-ready ecosystem that safely compresses these identifiers into the lightweight `Uint8Array` binary deltas required by the HTTP/3 streams.

3.7 Front-end Implementation

The front-end logic resides in `extension.js`. It utilizes the `vscode.workspace.onDidChangeTextDocument` event listener to capture local typing. When a user types, the changes are applied to the local `Y.Text` instance.

To minimize serialization overhead and reduce network bandwidth, the architecture intentionally avoids standard JSON payload transmission. Instead, Yjs immediately fires an `update` event, generating a highly compressed `Uint8Array` containing the mathematical delta of the edit. This binary payload is sent to the Webview Proxy, which pushes it over the HTTP/3 reliable stream to the server.

Incoming remote changes trigger the `vscode.window.activeTextEditor.edit()` builder to inject the collaborator's text onto the screen. However, because

programmatic injections inherently trigger the local `onDidChangeTextDocument` listener, the system implements a `remoteUpdateLock` mechanism. This integer-based lock ensures that remote edits are explicitly ignored by the outgoing broadcaster, successfully preventing infinite network feedback loops.

3.8 Back-end Implementation

Unlike traditional collaborative editors that rely on heavy, centralized databases to sequence text operations, the back-end server (`http3-server.mjs`) acts strictly as a lightweight, peer-to-peer router. Because the CRDT engine handles all mathematical conflict resolution on the client side, the server's only responsibility is the rapid, low-latency distribution of binary payloads.

To achieve this, the architecture abandons standard TCP WebSockets and replace it with HTTP/3. Because the core Node.js environment does not yet natively support these experimental APIs, the system utilizes the `@fails-components/webtransport` library to establish the connection.

Operating over the QUIC protocol allows the server to utilize multiplexing to separate traffic types, resolving the Head-of-Line (HoL) blocking issues that traditionally freeze collaborative interfaces, as shown in Figure 3.2:

- **Reliable Streams:** Used for critical `Y.Doc` state updates (typing, creating files, deleting files). If a network packet drops, QUIC retransmits it exclusively on this specific stream without pausing the rest of the application's traffic.
- **Unreliable Datagrams:** Used for the Awareness protocol (cursor movements). Datagrams are fired immediately without delivery guarantees. If a cursor packet drops, the system ignores it, ensuring that ephemeral UI doesn't bottleneck the critical document state.

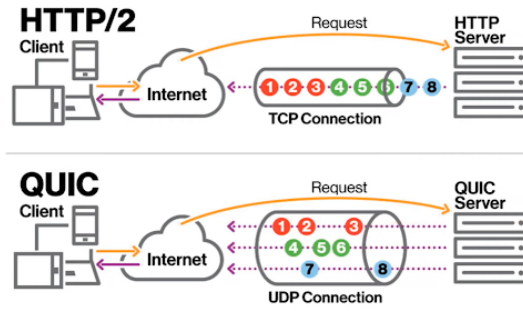


Figure 3.2: Comparison of connection establishment times for different network protocols [datadaddi2022].

3.9 Architectural and Security Challenges

The most significant engineering challenge in this project during development involved bypassing the strict security models enforced by the Electron framework (which powers VS Code).

3.9.1 The Sandbox and Private Network Access (PNA)

Beyond the inherent restrictions of the Electron sandbox, the project faced unforeseen roadblocks due to the rapidly evolving WebTransport specification. In April, a new draft update altered the certificate verification requirements for the HTTP/3 QUIC handshake within Chromium. This update suddenly deprecated the existing fallback mechanisms for self-signed development certificates, causing handshake failures. Initially, the back end was designed to run on the developer's local machine (`localhost:4433`) using self-signed OpenSSL certificates. However, the Electron Webview strictly enforces Chromium's **Private Network Access (PNA)** policy.

When the Webview attempted to initiate the UDP WebTransport handshake, the sandbox detected a cross-origin request to a local network that lacked specific PNA headers. Furthermore, the internal `X509Certificate` verification engine outright rejected the self-signed certificates (`CERTIFICATE_VERIFY_FAILED`). The Webview ignored developer flags (like `-ignore-certificate-errors`) to prevent malicious extensions from overriding core security protocols. This necessary pivot directly impacted the project timeline, limiting the capacity to implement further programmatic enhancements.

Thesis Decision Log: The Cloud Deployment Pivot

- **The Scenario:** The local VS Code sandbox actively killed the WebTransport cryptographic handshake.
- **The Decision:** Abandoned the `localhost` environment and deployed the back end to a cloud droplet attached to a dynamic DNS provider (`DuckDNS`).
- **Why this was chosen:** Moving the server to a public IP bypassed the PNA local-network restrictions. By provisioning a globally trusted `Let's Encrypt`[letsencrypt] SSL certificate via `certbot`, the server provided a valid Certificate Authority (CA) chain. The Electron sandbox recognized the public domain and valid cryptography, allowing the 1-RTT QUIC handshake to complete successfully.

3.9.2 Presence Data and Ephemeral State Garbage Collection

Managing user presence over WebTransport datagrams presented a challenge for tracking when users go offline. Because datagrams do not guarantee delivery, a sudden disconnection (such as a Wi-Fi failure or closing the app) happens without sending a proper "disconnect" message. As a result, the disconnected user's cursor gets left behind in the document's memory, creating a frozen "ghost cursor" on the screen. To solve this, the system uses an automatic cleanup timer (Time-To-Live). If it stops receiving regular signals from a user within a specific time limit, the system assumes they disconnected and safely removes their cursor.

Thesis Decision Log: Ephemeral State Lifecycle

- **The Scenario:** Handling abrupt client disconnections without explicit termination signals over UDP datagrams.
- **The Decision:** Implemented a Time-To-Live (TTL) heartbeat timeout within the Yjs Awareness protocol.
- **Why this was chosen:** Because datagrams are inherently unreliable, the system cannot guarantee a disconnect message will ever arrive. A TTL mechanism ensures that stale presence data (ghost cursors) is separately

garbage-collected by the remaining peers, maintaining UI accuracy without requiring a central server to track connection states.

3.9.3 CRDT Tombstone Accumulation and Memory Bloat

While Conflict-Free Replicated Data Types (CRDTs) successfully resolve concurrency without a central server, they introduce a structural memory problem. To guarantee Strong Eventual Consistency, the `Y.Text` data structure cannot permanently erase deleted characters; instead, it marks them with cryptographic "tombstones". In long-running collaborative sessions involving massive code replacements, this metadata accumulates, causing the underlying Abstract Syntax Tree (AST) to grow significantly larger than the visible text buffer. Managing this memory bloat and optimizing the serialization of the `Uint8Array` state updates was a necessary architectural consideration to prevent the VS Code Extension Host from experiencing memory exhaustion during extended use.

Thesis Decision Log: Tombstone Memory Management

- **The Scenario:** Preventing memory exhaustion and network degradation caused by the infinite accumulation of CRDT deletion tombstones.
- **The Decision:** Made two connected choices to handle data synchronization. First, to prevent the syncing logic from breaking, deleted text is hidden rather than truly erased, which causes the document's background memory to grow. Second, to prevent this growing file from slowing down the network, the app only transmits the exact edits made by the user, compressing them into tiny binary packets (`Uint8Array`) for maximum speed.
- **Why this was chosen:** Attempting to permanently erase deleted text from the data structure could permanently break the document if the internet connection drops. Relying on compressed binary deltas ensures that even as the internal document footprint grows, the network payloads remain lightweight, preserving low latency editing.

3.9.4 Middleware Lifecycle and Zombie Processes

Separating the network logic into a hidden Webview created a problem with how the application shuts down. The VS Code Extension and the hidden Webview run as completely separate background processes. Because of this, if the main extension crashes or is restarted by the developer, the hidden Webview might stay alive in the background as a "zombie" process. This window silently drains computer memory and internet bandwidth by keeping its server connection open. To prevent these memory leaks, the system is engineered with a strict communication link that forces the hidden Webview to completely shut down the exact moment the main extension closes.

Thesis Decision Log: Middleware Process Isolation

- **The Scenario:** Ensuring the hidden Chromium Webview proxy terminates cleanly when the VS Code Extension Host shuts down or crashes.
- **The Decision:** Bound the Webview lifecycle explicitly to the extension's internal deactivation hooks and context subscriptions.
- **Why this was chosen:** Decoupling the network logic into an isolated proxy creates the risk of "zombie" background processes. By forcing the Webview to share the extension's strict garbage collection lifecycle, the architecture guarantees that all orphaned HTTP/3 connections are severed and memory leaks are prevented upon application closure.

3.10 Security and Access Control

Unlike traditional centralized web applications that rely on complex user databases, this collaborative extension utilizes an ephemeral, peer-to-peer security model.

3.10.1 Role Specifications

Authorization is enforced through session-based roles rather than persistent database accounts. The system simplifies users into two functional categories, as shown in both diagrams Table 3.1 and Figure 3.3:

3.11 Testing and Validation

Due to the real-time, networked nature of the application, standard REST API testing methodologies were insufficient. The testing strategy focused on data convergence (CRDT testing) and network resilience.

3.11.1 Unit Testing (CRDT State)

Unit testing focused on the Yjs data structures to ensure that concurrent document mutations did not result in corrupted Abstract Syntax Trees (ASTs) or lost characters.

- **Convergence Testing:** Simulated multiple users inserting and deleting text at the exact same index simultaneously to verify the mathematical conflict resolution of the `Y.Text` object.
- **Awareness Garbage Collection:** Verified that when a user's connection drops unexpectedly, their cursor coordinates and presence data are successfully purged from the `awarenessProtocol` state within a 30-second timeout window.

3.11.2 Integration Testing (The Proxy Pipeline)

Integration tests validated the internal message-passing pipeline between the VS Code Extension Host and the Webview Proxy.

- **Payload Encoding:** Ensured that the `Uint8Array` binary updates generated by Yjs were correctly serialized into the `postMessage` API without data corruption.
- **Handshake Verification:** Validated that the Webview successfully negotiated the 1-RTT TLS 1.3 handshake with the `Let's Encrypt`[letsencrypt] secured cloud server.

3.11.3 Manual UI and Network Testing

Extensive manual testing was conducted to evaluate the system under simulated real-world network conditions.

- **High Latency Simulation:** Used network throttling tools to simulate a 300ms delay. Verified that local typing remained fast(almost zero UI lag) while remote changes arrived smoothly.
- **Workspace Synchronization:** Verified that creating, editing, and deleting a shared file instantly updated the Custom TreeView Explorer for all connected peers.
- **Disconnection Handling:** Confirmed that clicking the "Disconnect" button successfully terminated the QUIC stream, removed the user's remote cursors from other screens, and closed the Webview proxy.

User Stories

Goal	Given	When	Then
Host a session	Command Palette is running(by pressing Ctrl+Shift+P).	The user types and selects the command "HTTP/3 Yjs: Manage Session" then select the "Create New Room" option and give my user name.	A Room ID is generated, copied to the clipboard, and the proxy panel opens.
Join a session	Command Palette is running(by pressing Ctrl+Shift+P).	The user types and selects the command "HTTP/3: Manage Session" then selects the "Join Existing Room" option and provides their user name and the provided room id.	The network proxy panel will open and the user will instantly receive all the shared files in the file explorer.
Open the shared file explorer	A session is on and Command Palette is running(by pressing Ctrl+Shift+P)	The user types and selects the command "View: Show HTTP/3 Collab".	A new "Shared Session Explorer" menu appears where all the files created by the other users in the session
Create a shared file	A session is on and the shared file explorer is open.	The user types and selects the command "HTTP/3 Yjs: Create Shared File" and enters the name of the shared file.	It will appear instantly in everyone's Shared Session Explorer and can be accessed by all users in the session and edit it synchronously.
Edit a shared file	A session is on and a shared file is open from the shared session explorer.	The user edits the file by typing or moving their cursor around the screen in the file.	Changes appear in the screen of all connected users to the session asynchronously.
Disconnect from session	A session is on.	The user presses the disconnect button located inside the Network Proxy sidebar panel.	The HTTP/3 connection closes securely and erase the user's cursor from other user's screens.
Deleting a file from the session	A session is on and a file is created	The user hovers over the file in the file explorer and, clicks the trash icon next to it.	The file disappear from the shared file explorer of all users.

Table 3.2: Core User Workflows

3.12 Chapter Conclusion

By the end of this chapter, a future developer should possess a comprehensive understanding of the architectural foundation underlying this collaborative extension. Building a real-time, peer-to-peer text editor within the strict confines of the Visual Studio Code environment required and abandoning traditional centralized, TCP-based architectures.

Instead, this implementation successfully integrates three distinct engineering solutions:

1. **CRDT State Management:** Replacing absolute indexing with the Yjs mathematical model to guarantee Strong Eventual Consistency without a central server.
2. **HTTP/3 and WebTransport:** Eliminating TCP Head-of-Line blocking by splitting the network traffic. Large document updates and lightweight cursor movements are sent through independent QUIC streams, ensuring one dropped packet never halts the entire system.
3. **Decoupled Cloud Proxy:** Bypassing the Node.js limitations and Chromium Private Network Access (PNA) sandboxing by offloading network handshakes to a hidden Webview, and then secure a connection that is verified by a public cloud Certificate Authority.

By resolving these core concurrency and networking challenges, the resulting architecture is resilient and performant. A developer is now equipped to navigate the source code, comprehend the underlying conflict-resolution mechanics, and confidently extend the project's features in future iterations.

Conclusion

This chapter provides a quick summary of how the project met its original goals. It also discusses future improvements for the application and ends with my personal takeaways from the development process.

Achievement of Objectives

The main goal of this thesis was to build a fast, real-time collaborative text editor directly inside Visual Studio Code. Traditional collaborative tools rely on standard TCP connections, which can cause the screen to freeze or lag when the network is unstable due to Head-of-Line blocking. TCP suffers from Head-of-Line (HoL) blocking, which describes an occurrence during congestion control when a specific data packet is stalled on the way, leading to all following packets being stalled as well. The QUIC protocol addresses many of these issues by streamlining the opening handshake, having security in mind from the start, as well as having vastly improved congestion algorithms that completely eliminate HoL blocking.[[nguyen2024](#)]

To solve this, the project successfully built a new architecture using HTTP/3 and WebTransport. This allowed large code updates and fast mouse cursor movements to travel on separate network streams.

Additionally, the project used Conflict-Free Replicated Data Types (CRDTs) through the Yjs library. Operating via an indirect transformation approach, this allowed multiple users to edit the exact same file concurrently without relying on a centralized sequencing server [[sun2020](#)]. Finally, the system successfully bypassed VS Code's strict security rules by creating a hidden browser window (proxy) and connecting to a secure cloud server.

Future Considerations

While the current prototype works well, there is room for future improvements. For example, the core Node.js environment that powers VS Code does not support WebTransport natively yet. Once Node.js adds official support for it in the future, the hidden browser window can be completely removed to make the extension lighter and faster. Other features, such as built-in audio chat or a more advanced way to clean up the hidden deleted text (CRDT tombstones), could also be added to improve performance during very long coding sessions.

Concluding Remarks

Building this project was a challenging but very rewarding experience. It required combining experimental networking protocols(QUIC) with advanced mathematical data structures(CRDT). I gained a much deeper understanding of distributed systems, network security, and how to build complex extensions for Visual Studio Code. This project serves as a great showcase of the knowledge I have gathered during my Computer Science BSc studies, and provides a strong foundation for my future career in software engineering.

Bibliography

- [1] D. Sun, C. Sun, Agustina, and W. Cai, “Real Differences between OT and CRDT in Building Co-Editing Systems and Real World Applications,” *CoRR*, vol. abs/1905.01517, 2019. Available: <http://arxiv.org/abs/1905.01517>
- [2] V. M. Antonova, I. G. Buzhin, N. A. Grechishkina, E. D. Korochkin, and N. A. Kuznetsov, “QUIC Transport Protocol as a Reliable Transmission Method for Real-Time Traffic,” *Journal of Communications Technology and Electronics*, vol. 70, no. 8, pp. 410–417, 2025.
- [3] V. B. F. Gomes, M. Kleppmann, D. P. Mulligan, and A. R. Beresford, “Verifying strong eventual consistency in distributed systems,” *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 1–28, 2017.
- [4] M. Nguyen, P. Nys, S. Pham, D. Silhavy, S. Arbanowski, and S. Steglich, “Toward WebTransport Support in HTTP Adaptive Streaming,” in *2024 Tenth International Conference on Communications and Electronics (ICCE)*, 2024, pp. 96–101.
- [5] O. M. Bondar, “Analysis of Next-Generation Internet Transport Protocols: QUIC, WebTransport, HTTP/3,” *Information Technologies and Systems*, vol. 6, no. 6, pp. 52–63, 2025.
- [6] TheDataDaddi, “The QUIC Protocol, HTTP3, and How HTTP Has Evolved,” YouTube, Feb. 28, 2022. [Video]. Available: https://www.youtube.com/watch?v=VONSx_ftkz8
- [7] Internet Security Research Group (ISRG), “Let’s Encrypt: A Free, Automated, and Open Certificate Authority,” 2026. [Online]. Available: <https://letsencrypt.org/>

- [8] Chromium Blog (A. Wilk, R. Hamilton, I. Swett), “A QUIC update on Google’s experimental transport,” Apr. 17, 2015. [Online]. Available: <https://blog.chromium.org/2015/04/a-quic-update-on-googles-experimental.html>